MACHINE LEARNING IN PHYSICS
GRAPH AUTOENCODERS
PART 1

HARRISON B. PROSPER

PHY6937 / PHY4936

Outline

- Graphs Revisited
- Graph Autoencoder – Part 1

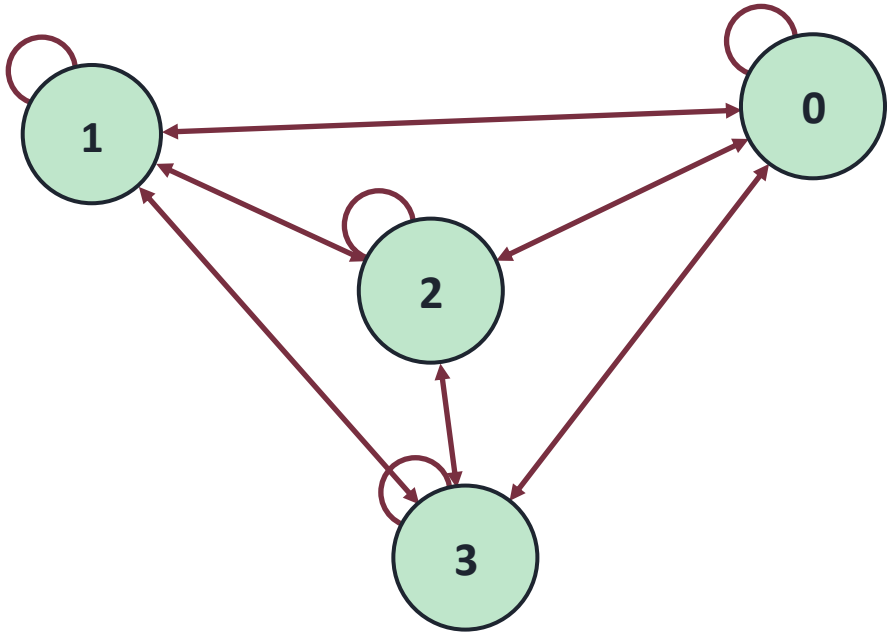
Lecture based on notes by Dr. Claire David

African Institute of Mathematical Sciences, Cape Town, South Africa.

Graphs Revisited (1): Graph

- A **graph** $G = (V, E)$ comprises **nodes** (or vertices) V and **edges** E that model the connections between nodes.

- Here is a **fully-connected graph** containing 4 nodes and 16 edges*.

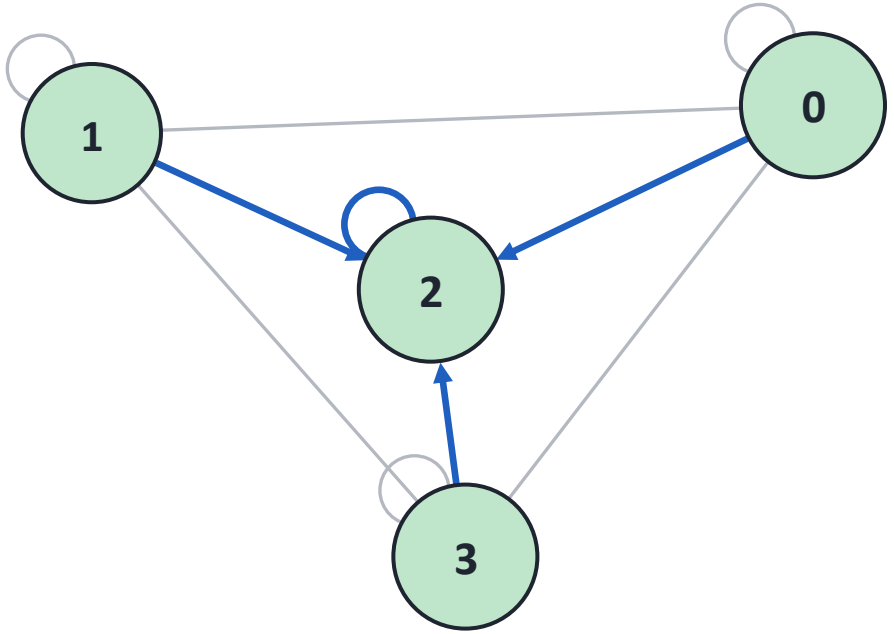


* 6 of the edges are doubled.

Graphs Revisited (2): Message

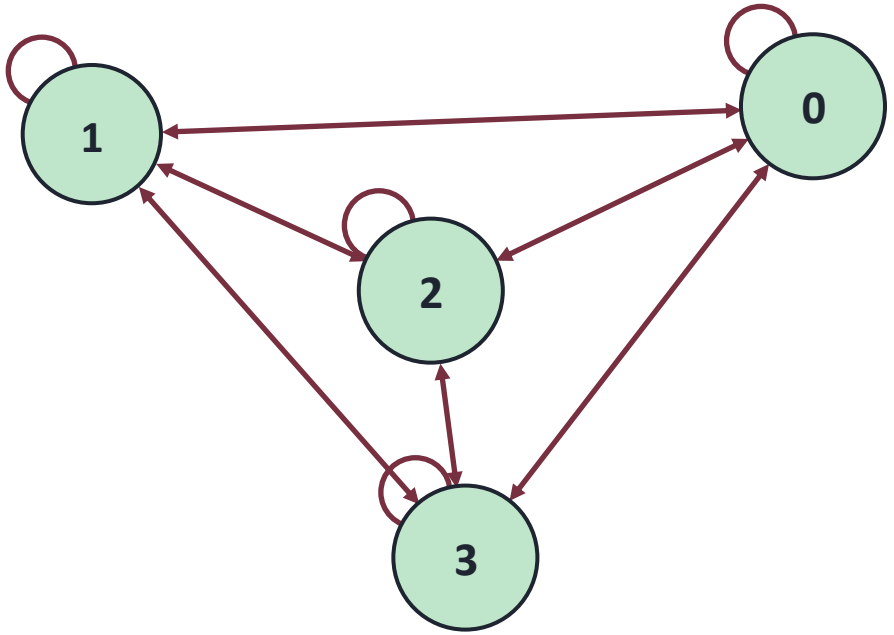
- A **message** is data passed from a **source node** and received by a **target node**.

- Example: Node **2** is a target node, which receives messages from the source nodes 0, 1, 3, and from itself.



Graphs Revisited (3): Index

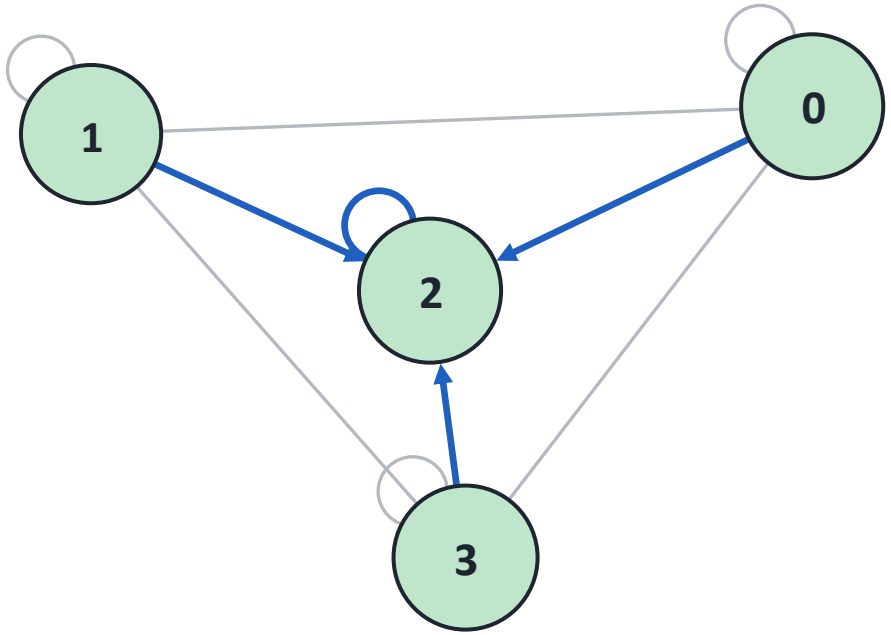
- A node has a **node index** i and a **feature vector** v.
- An edge has an **edge index** (i, j) and feature vector e; i is the target node index and j the index of the source node.
- A message is a function $M = g(v, e)$ of v and e.



Graphs Revisited (4): Aggregation

- The messages received are processed through an **aggregation** function, which outputs a vector of attributes that replace the existing attributes of the target node.

- Consider node 2. It receives messages from nodes 0, 1, 3 and from itself and uses these messages to update its own attributes.

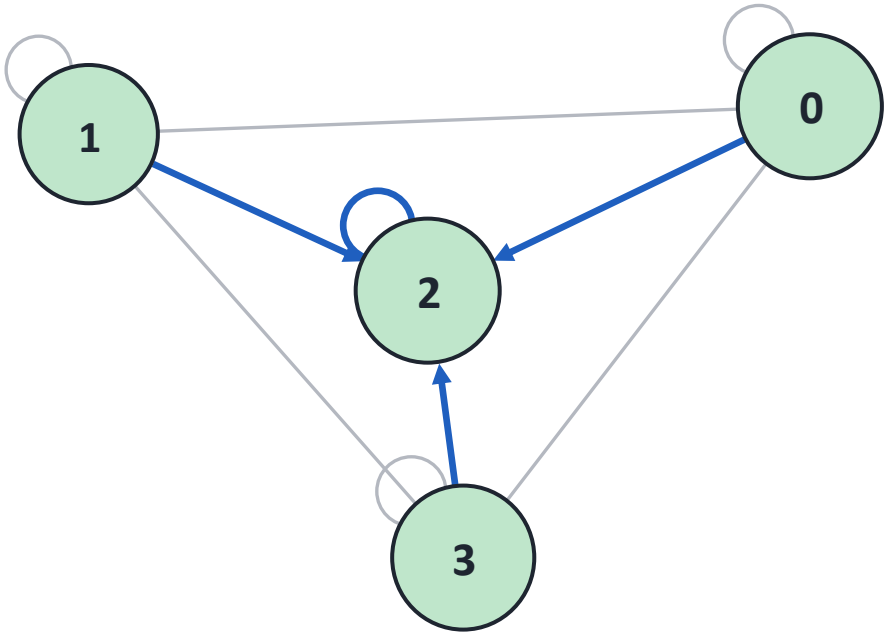


- A typical aggregation is just a vectorial sum of the messages.

Graphs Revisited (5): Graph Conv

- Aggregation in graphs is the analog of convolution in convolutional neural networks and, therefore, it is referred to as **graph convolution**.

- In a CNN, the analog of the source nodes are the nearest neighbor pixels of a given
  pixel, while the given
  pixel is the analog of the
  target node.

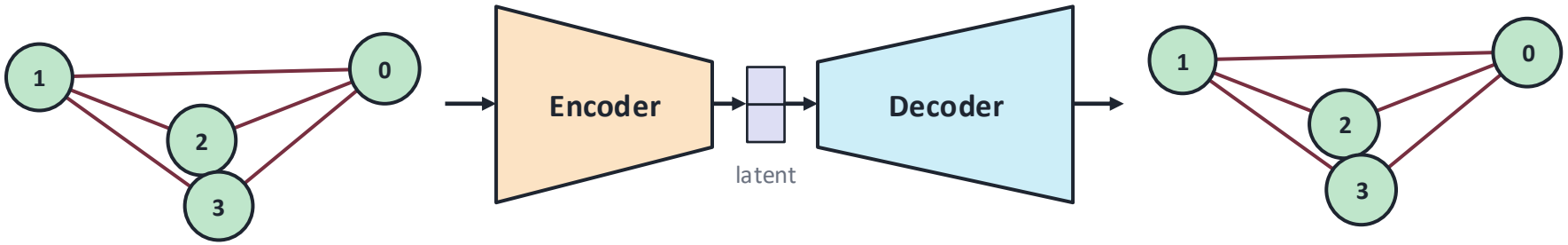


GRAPH AUTOENCODER

PART 1

Graph Autoencoder (1)

- A graph autoencoder (GAE) is a function that *encodes* a graph in a latent space and *decodes* the latent space representation of the graph back to a graph, e.g., the original.



- A typical application is detecting anomalies in a graph: if a GAE fails to reconstruct the input graph correctly, this may indicate an anomaly. However, this claim has recently been called into question[1].

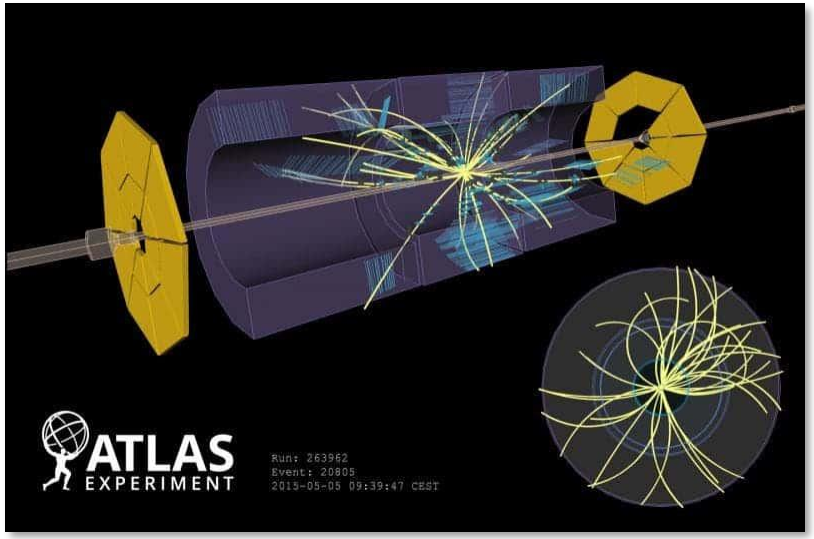
1. Autoencoders for Anomaly Detection are Unreliable, Roel Bouman, Tom Heskes, https://arxiv.org/abs/2501.13864

Graph Autoencoder (2)

- To make our discussion concrete, let's use the same data we used for the graph neural network, namely, particle physics data.

$$S = \{ (p_T, \eta, \phi)_i \}_{i=1}^{n}$$

- We'll build an autoencoder that accepts a tensor of nodes, V (or vertices) of shape $(n, 3)$, where each row represents a particle with 3 features, p_T, η, ϕ.



Graph Autoencoder (3): Dataset

Nodes

p_{T0}	η_0	ϕ_0
p_{T1}	η_1	ϕ_1
p_{T2}	η_2	ϕ_2
p_{T3}	η_3	ϕ_3

$V[n = 4,\ d = 3]$

Edge index

trg	0	0	0	0	⋯	3
src	0	1	2	3	⋯	3

edge_index$[2,\ K = 16]$

K is the number of edges.

d is the dimension of the input feature vectors.

target node index (trg) = edge_index$[0]$

source node index (src) = edge_index$[1]$

Graph Autoencoder (4): Embedding

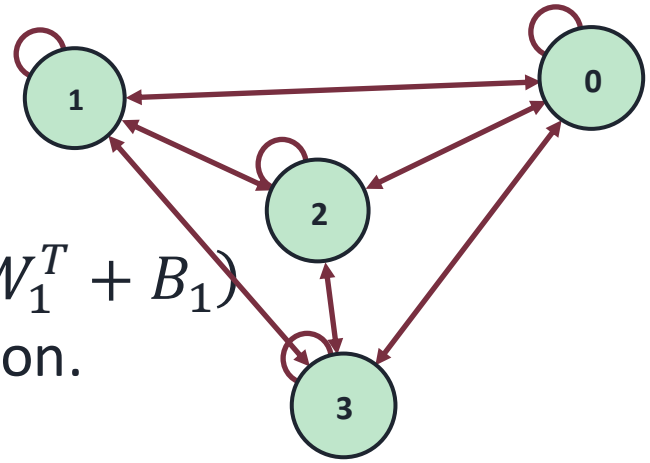
As we have seen, it is common practice in many ML models to map the input data into an **embedding space** (of dim $= h$)

V input nodes

p_{T0}	η_0	ϕ_0
p_{T1}	η_1	ϕ_1
p_{T2}	η_2	ϕ_2
p_{T3}	η_3	ϕ_3

X (embedded nodes)

a_{00}	a_{01}	$\cdots$	a_{0h-1}
a_{10}	a_{11}	$\cdots$	a_{1h-1}
a_{20}	a_{21}	$\cdots$	a_{2h-1}
a_{30}	a_{31}	$\cdots$	a_{3h-1}



$$X = \sigma(VW_1^T + B_1)$$

W_1 and B_1 are matrices and σ a non-linear function.

Graph Autoencoder (5): Edge Feature

In our example, each edge is associated with a single scalar:

$$e_{(i,j)} = \exp\left[\exp\left(-\alpha\left\|x_i - x_j\right\|^2\right)\right] / \sum_{k=0}^{n-1} \exp[\exp(-\alpha\|x_i - x_k\|^2)]$$

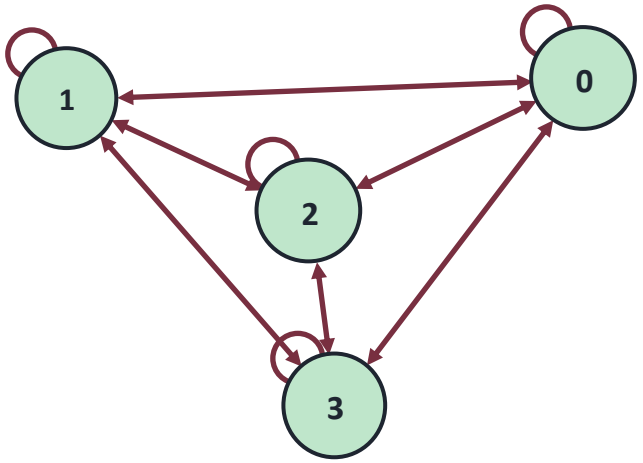
where α is a learnable parameter.

$$x_i = (\eta_i, \phi_i)$$

$$e_{(i,j)} = \mathrm{softmax}\left[\exp\left(-\alpha\left\|x_i - x_j\right\|^2\right)\right]$$

(16) edge features

$\boldsymbol{e_{(0,0)}}$	$\boldsymbol{e_{(0,1)}}$	$\boldsymbol{e_{(0,2)}}$	$\boldsymbol{e_{(0,3)}}$	$\cdots$	$\boldsymbol{e_{(3,3)}}$

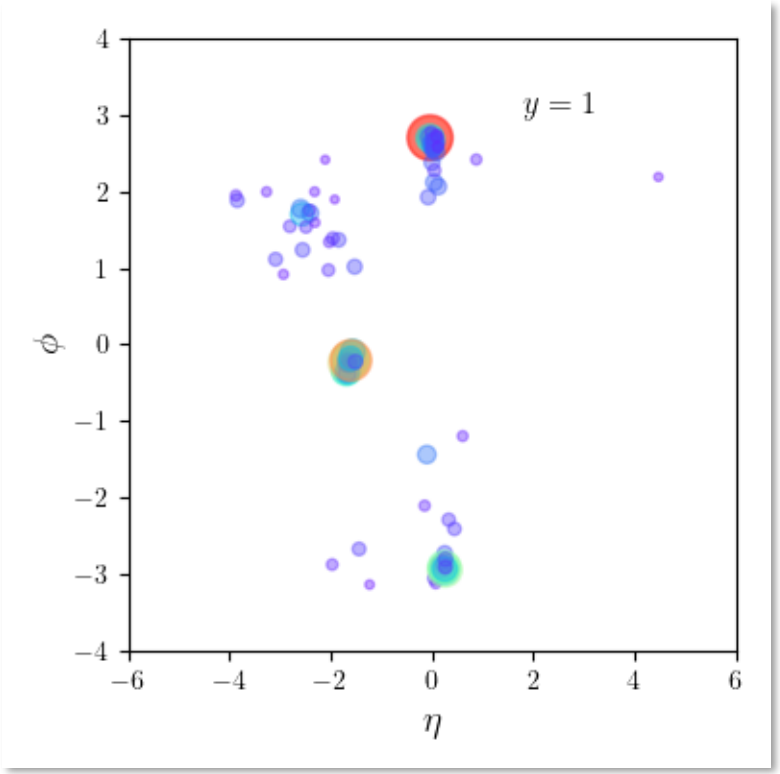


Graph Autoencoder (6): Edge Feature

- The edge features can be interpreted as **weights** that quantify how strongly two nodes (i.e., particles) are related.

- Particles that are close together in (η, ϕ)-space have larger weights than those that are further apart.

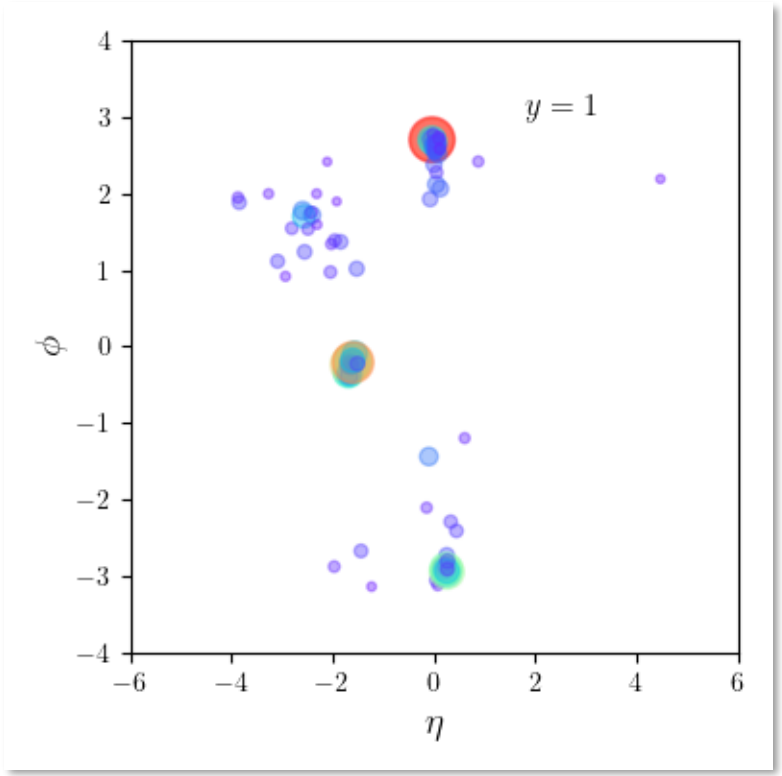
- In more sophisticated GAEs, each edge is usually associated with several edge features.



Graph Autoencoder (7): Adjacency

- The weights can be thought of as an $n \times n$ **adjacency matrix** A that defines the **neighborhood structure** of the graph. Each row corresponds to a target node and each column to a source node.
- Example: target node 0 is strongly related to itself and source nodes 1 and 3 but not to source node 2.

$e_{(0,0)}$	$e_{(0,1)}$	$e_{(0,2)}$	$e_{(0,3)}$
$e_{(1,0)}$	$e_{(1,1)}$	$e_{(1,2)}$	$e_{(1,3)}$
$e_{(2,0)}$	$e_{(2,1)}$	$e_{(2,2)}$	$e_{(2,3)}$
$e_{(3,0)}$	$e_{(3,1)}$	$e_{(3,2)}$	$e_{(3,3)}$



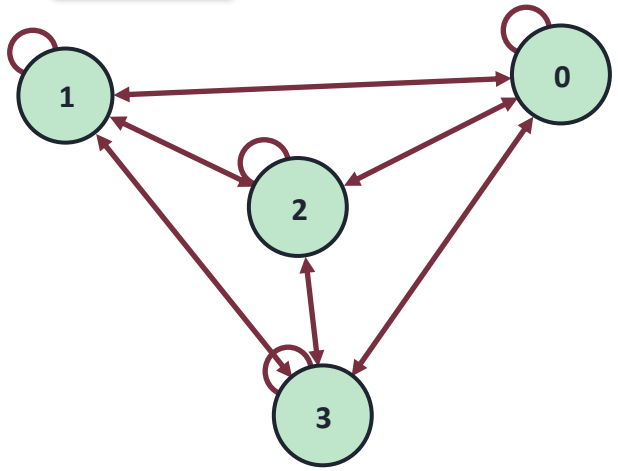
Graph Autoencoder (8): Adjacency

- Suppose X is a matrix, of shape $[n, h]$, of embedded nodes. Consider the product AX and focus on one target node (a row) and one feature (a column) of X.

$\boldsymbol{e}_{(2,0)}$	$e_{(2,1)}$	$\boldsymbol{e}_{(2,2)}$	$\boldsymbol{e}_{(2,3)}$

a_{00}
a_{10}
a_{20}
a_{30}

- The matrix multiplication computes a dot product between each column of X (which, as noted, corresponds to an embedded feature) and each row of the adjacency matrix A. In the example, only source nodes 0, 2, and 3 contribute substantially to the dot product.



To be continued…!

